Student & Course Enrollment System

Tables:

1. students

```
CREATE TABLE students (
  student_id INT PRIMARY KEY,
  name VARCHAR(50),
  department VARCHAR(50)
);

INSERT INTO students VALUES
(1, 'Alice', 'CS'),
(2, 'Bob', 'IT'),
(3, 'Carol', 'CS'),
(4, 'David', 'ECE');
```

2. **courses**

```
CREATE TABLE courses (
  course_id INT PRIMARY KEY,
  course_name VARCHAR(50)
);

INSERT INTO courses VALUES
(101, 'DBMS'),
(102, 'Networks'),
(103, 'Operating Systems');
```

3. **enrollments**

```
CREATE TABLE enrollments (
  student_id INT,
  course_id INT,
```

```
  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (course_id) REFERENCES courses(course_id)
);

INSERT INTO enrollments VALUES
(1, 101),
(2, 101),
(2, 102),
(4, 103);
```

student_id	Name	department
1	Alice	CS
2	Bob	IT
3	Carol	CS
4	David	ECE

course_id	course_name
101	DBMS
102	Networks
103	Operating Systems

student_id	course_id
1	101
2	101
2	102
4	103

1. INNER JOIN (Students who are enrolled in a course)

```
SELECT s.name, c.course_name
FROM students s
INNER JOIN enrollments e ON s.student_id = e.student_id
INNER JOIN courses c ON e.course_id = c.course_id;
```

Result:

name	course_name
Alice	DBMS
Bob	DBMS
Bob	Networks
David	Operating Systems

2. LEFT JOIN (All students and the courses they're enrolled in, if any)

```
SELECT s.name, c.course_name
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id
LEFT JOIN courses c ON e.course_id = c.course_id;
```

Result:

name	course_name
Alice	DBMS
Bob	DBMS
Bob	Networks
Carol	NULL
David	Operating Systems

3. RIGHT JOIN (All courses and students enrolled, if any)

```
SELECT s.name, c.course_name
FROM enrollments e
RIGHT JOIN courses c ON e.course_id = c.course_id
LEFT JOIN students s ON e.student_id = s.student_id;
```

Result:

name	course_name
Alice	DBMS

name	course_name
Bob	DBMS
Bob	Networks
David	Operating Systems

4. FULL OUTER JOIN (Simulated using UNION of LEFT and RIGHT JOIN)

SELECT s.name, c.course_name

FROM students s

LEFT JOIN enrollments e ON s.student_id = e.student_id

LEFT JOIN courses c ON e.course_id = c.course_id

Name	course_name
Alice	DBMS
Bob	DBMS
Bob	Networks
Carol	NULL
David	Operating Systems
NULL	NULL

UNION

SELECT s.name, c.course_name

FROM courses c

LEFT JOIN enrollments e ON c.course_id = e.course_id

LEFT JOIN students s ON e.student_id = s.student_id;

Result:

name	course_name
Alice	DBMS
Bob	DBMS
Bob	Networks

name	course_name
Carol	NULL
David	Operating Systems
NULL	DBMS
NULL	Networks
NULL	Operating Systems

5. SELF JOIN (Find students in the same department)

```
SELECT A.name AS Student1, B.name AS Student2, A.department
FROM students A
JOIN students B ON A.department = B.department
WHERE A.student_id < B.student_id;
```

Result:

Student1	Student2	department
Alice	Carol	CS

Set operations:

6. UNION (All unique student names from CS and IT departments)

```
SELECT name FROM students WHERE department = 'CS'
UNION
SELECT name FROM students WHERE department = 'IT';
```

Result:

name
Alice
Carol
Bob

7. INTERSECT (using IN or INNER JOIN)

Find students who are in CS and enrolled in at least one course

```
SELECT DISTINCT s.name
FROM students s
INNER JOIN enrollments e ON s.student_id = e.student_id
WHERE s.department = 'CS';
```

(This simulates an INTERSECT between students in CS and those who have enrollments.)

8. MINUS (using NOT IN)

Find students who are not enrolled in any course

```
SELECT name
FROM students
WHERE student_id NOT IN (SELECT student_id FROM enrollments);
```

Result:

name
Carol

Summary

Concept	MySQL Support	Notes
INNER JOIN	Yes	Matches common rows
LEFT JOIN	Yes	All from left + matching right
RIGHT JOIN	Yes	All from right + matching left
FULL OUTER JOIN	(Simulated)	Use UNION of LEFT and RIGHT JOIN
SELF JOIN	Yes	Join table with itself using alias
UNION	Yes	Combines results vertically
INTERSECT	(Simulated)	Use INNER JOIN with conditions
MINUS	(Simulated)	Use NOT IN, LEFT JOIN WHERE NULL